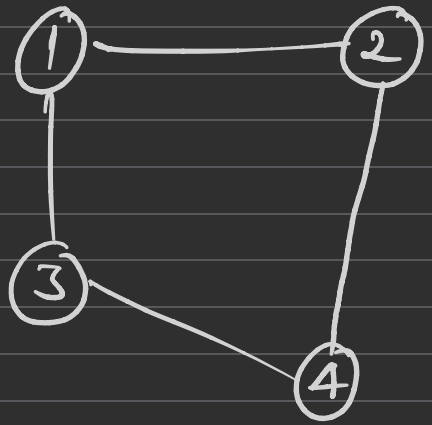
What is Graph

- Non-linear Data structure
- Nodes and Edges { Vertices and Edges }

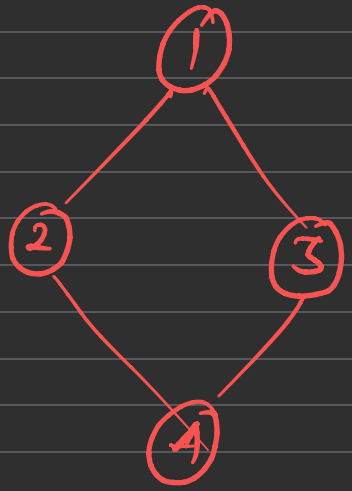
Collection of set of vertices (Nodes) and edges

$G \in (V, E)$

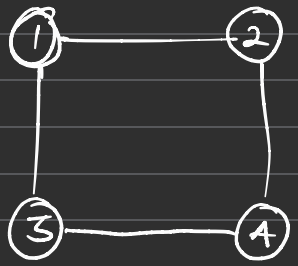


Tree (Never contain Cycle)

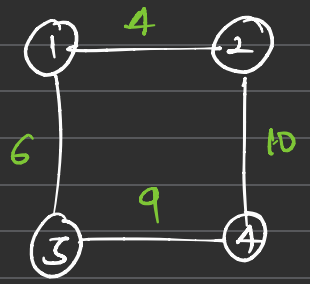
graph (can contain Cycle)



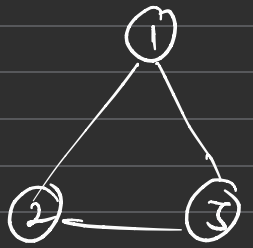
Types of Graphs



Simple graph

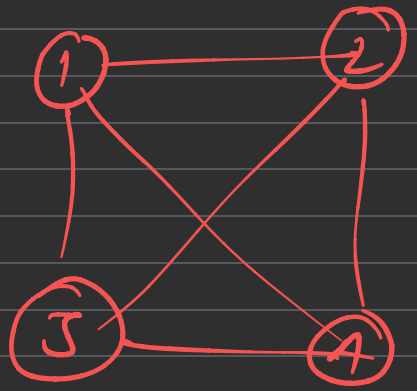


Weighted graph



Cycle graph

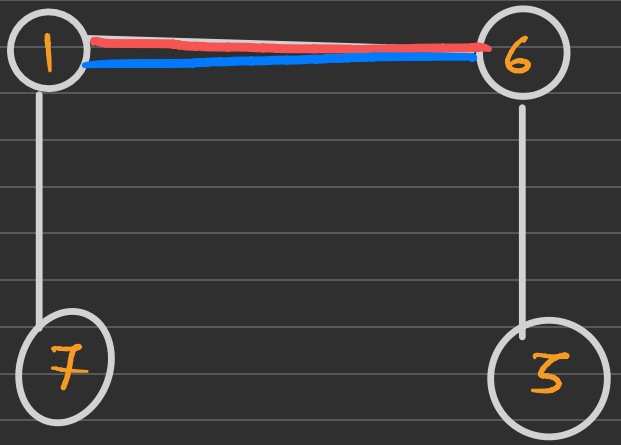
Complete graph



Directed Undirected Graph

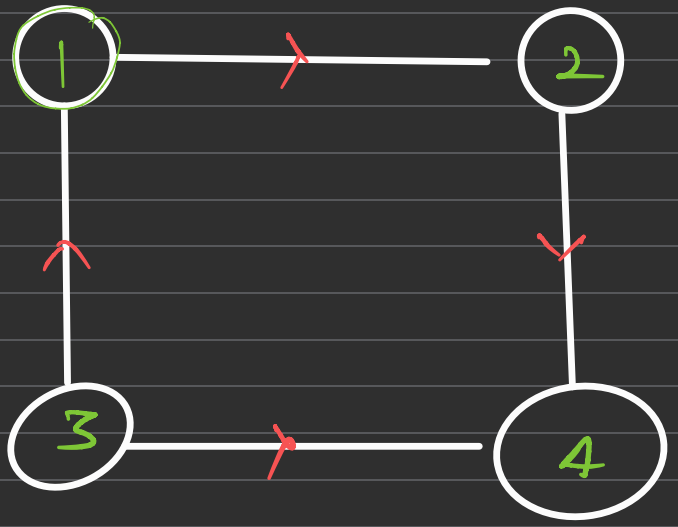
* Undirected graph

graph whose edge don't have particular direction



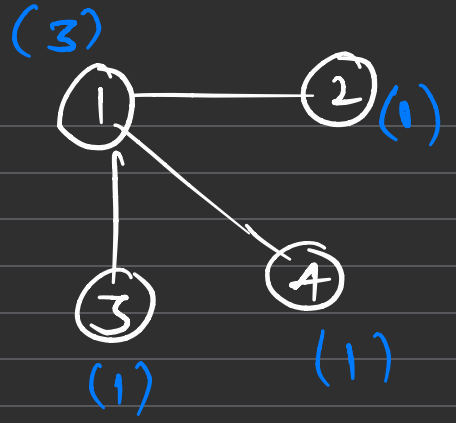
Directed Graph

graph whose edge have particular direction



* Degree (Undirected)

No. of edge connected to a Node

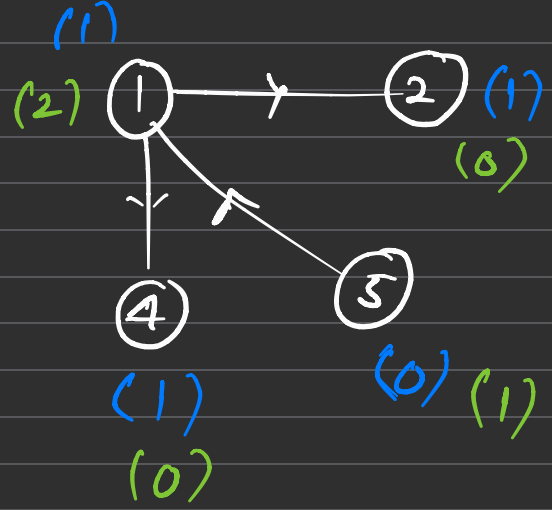


* Indegree (Directed)

No. of Edges comming inside a Node

* Outdegree (Directed)

No. of Edges going outside a Node



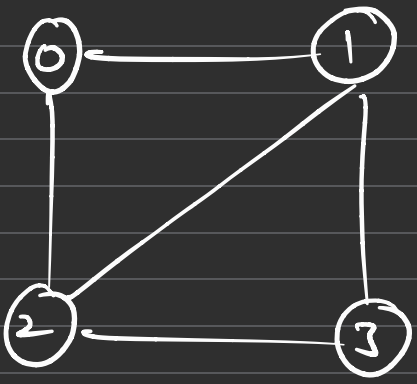
How to represent the Graph

- ~~Adj. Matrix~~
- ~~Adj. List~~

Adj. Matrix

V = 4

Build the matrix of VxV



Col

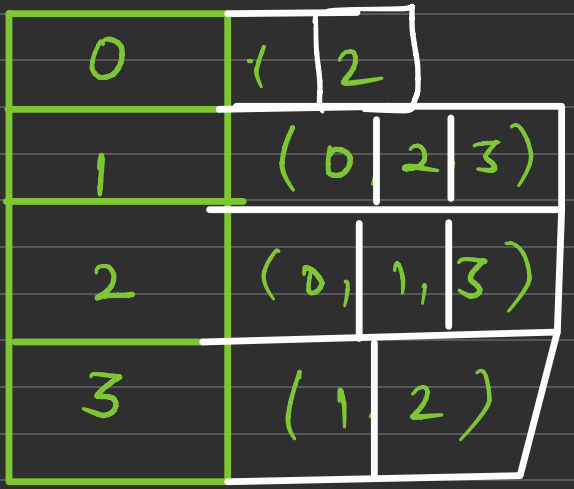
Row	0	1	2	3
0	0	1	1	0
1	1	0	1	1
2	1	1	0	1
3	0	1	1	0

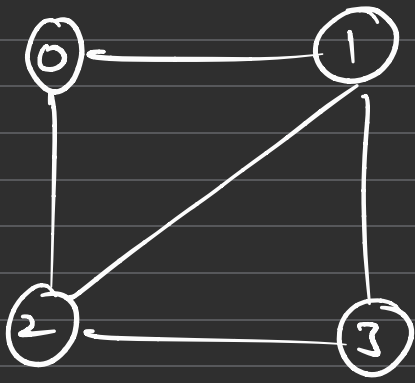
mat

mat[i][j] = 1
iff there is edge
from i to j
else
mat[i][j] = 0

Adj. List

V = 4





(0) → (1,2)

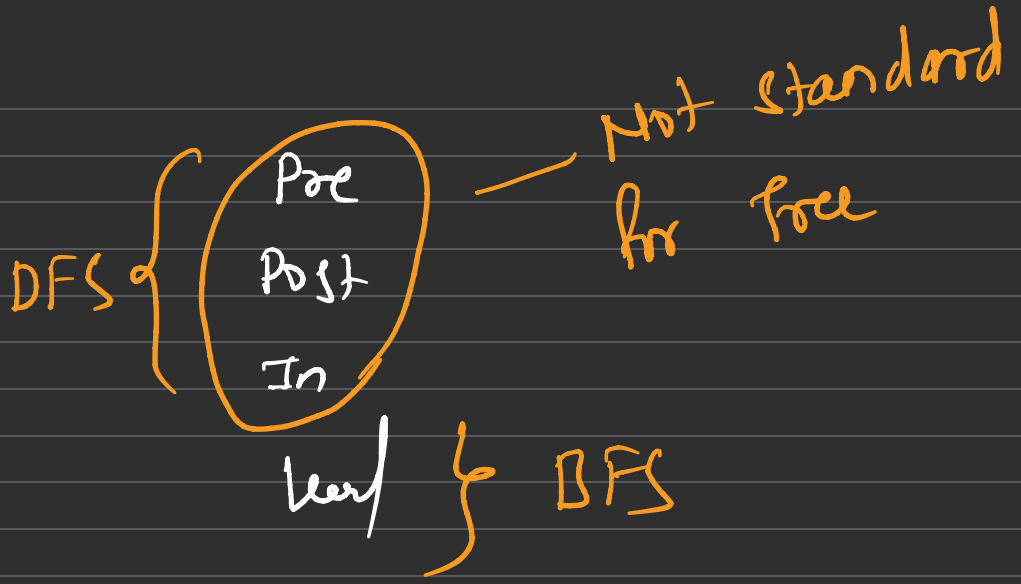
(1) → (0, 3,2)

(2) → (0, 1, 3)

(3) → (1,2)

Graph Traversal

- ~~DFS~~ ✓
- BFS



0	1	2	3
1	4	5	6
2	7	8	9

```
for (auto i : mat[index])
    print(i)
}
```
(cpp)

index = 1

```
for ( Integer i : mat.get(index))
    print(i)
}
```
(java)

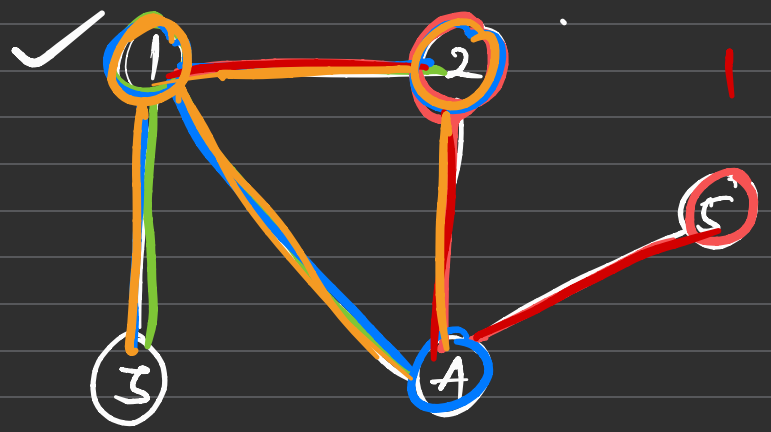
arr

6	7	2	1
0	1	2	3

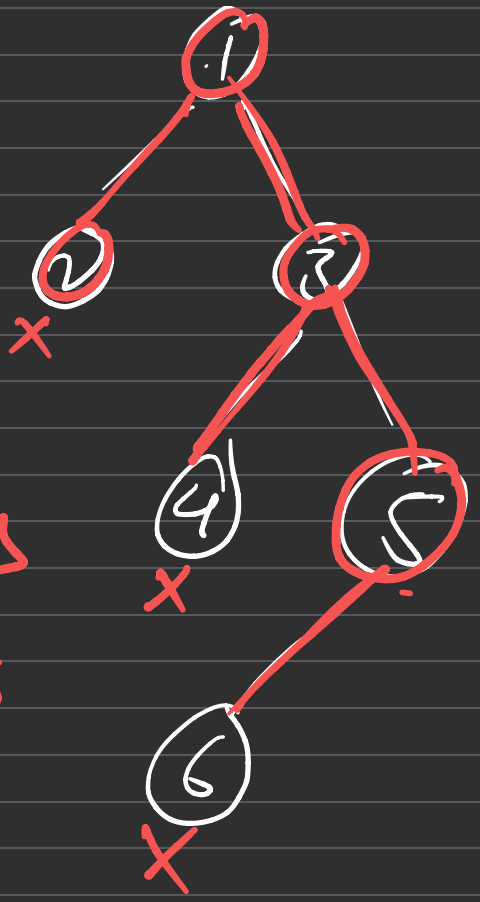
```
for (int i: arr)
    println(i)

for( auto it : arr )
    print (it)
```

print (arr(2)) = 2

1 2 3 4 1 4 5 1 2 3 4 Re



1 2 4



1

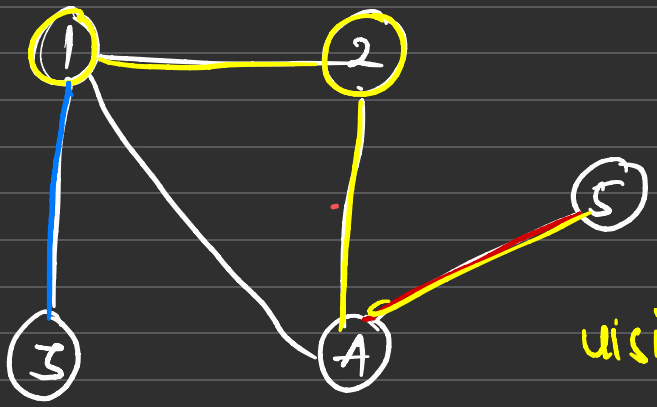
1 2 3

4 5 6

Stop

Stop point ?

Node Values = Index



we need to maintained array, (visited array)

visited

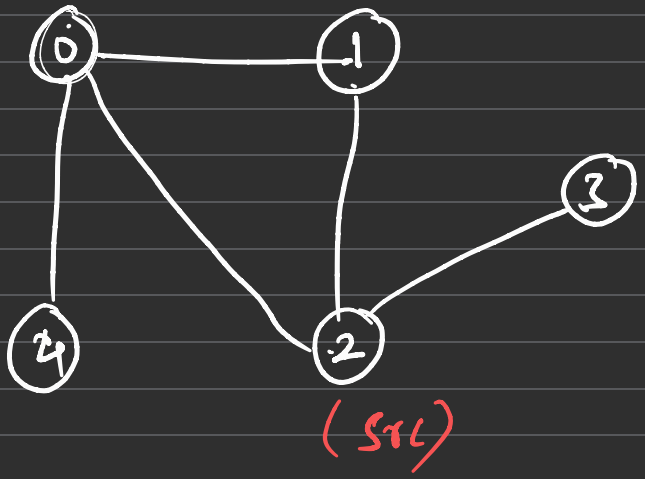
X	1	1	1	0	1
0	1	2	3	4	5

1 2 4 5 3

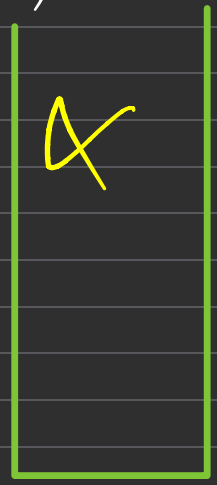
0 = Not visited

1 = visited

BFS (Breadth First Search → level order)



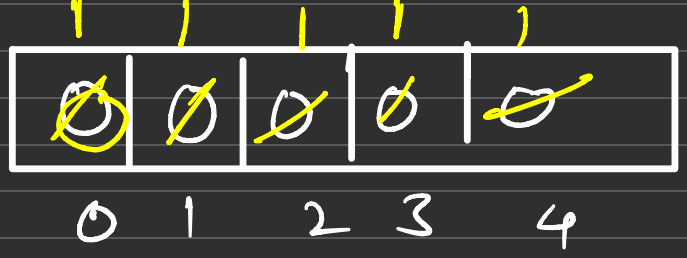
(src)



Queue<int>

2 0 1 3 4

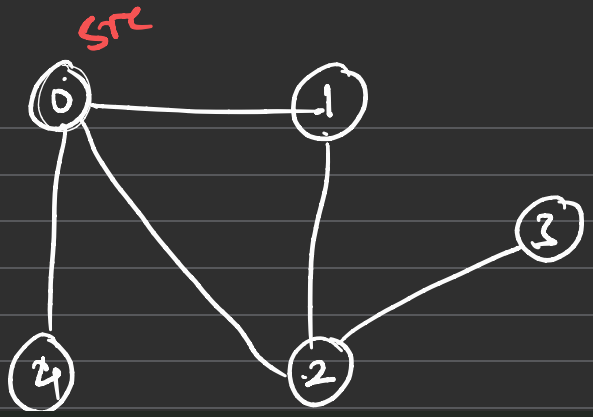
visit

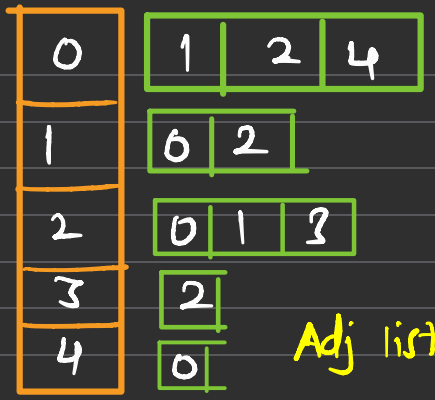


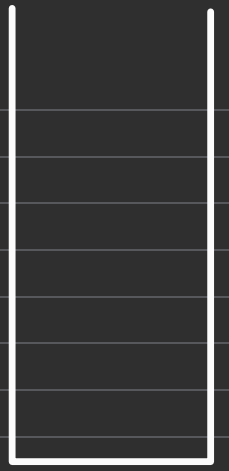
```
void BFS ( vector<int> adj[] , V )
{
    queue <int> q
    int visit [V] = {0}
    q.push (0)
    visit(0) = 1
```

```
while( !q.empty())
{   int node = q.front()
    q.pop()
    print (node)
    for (auto it : adj[node])
    {   if (visit[it] == 0)
        {   q.push (it)
            visit[it] = 1
        }
    }
}
```

1 2 4

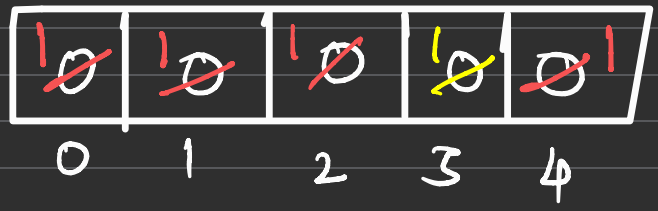






```
while(!q.empty()){
    int node = q.front();
    q.pop();

    ans.push_back(node);

    for (auto adjNode: adj[node]){

        // not visited
        if (visit[adjNode] == 0){
            q.push(adjNode);
            visit[adjNode] = 1;
        }
    }
}
return ans;
```

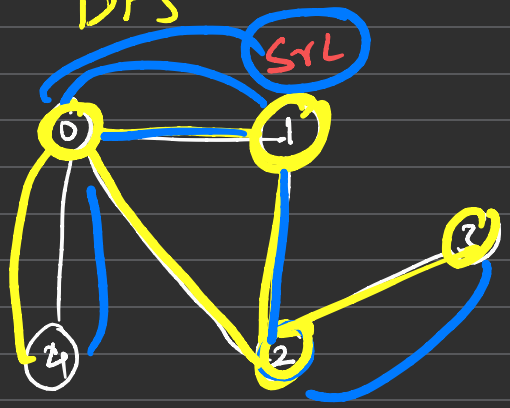
0 1 2 4 3

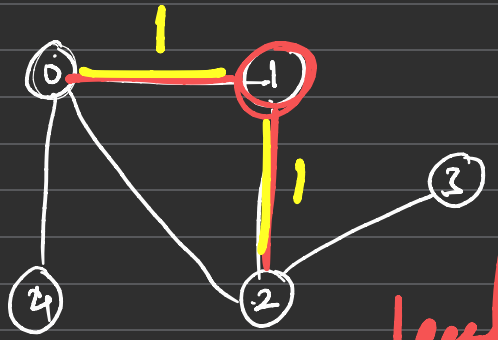


node = 3

adjNode = 2

DFS





0	1 → 2 → 4
1	0 → 2
2	0 → 1 → 3
3	2
4	0

Adj list

level

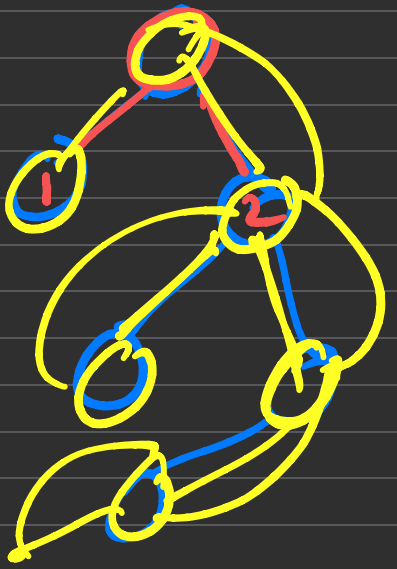
level

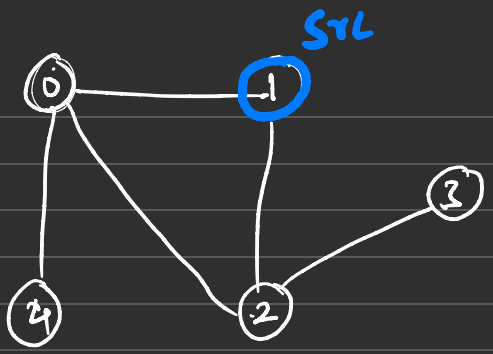
1 2

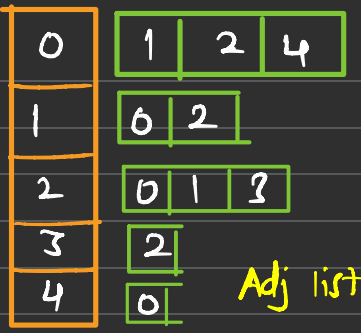
2 1

1 2 3 0 4

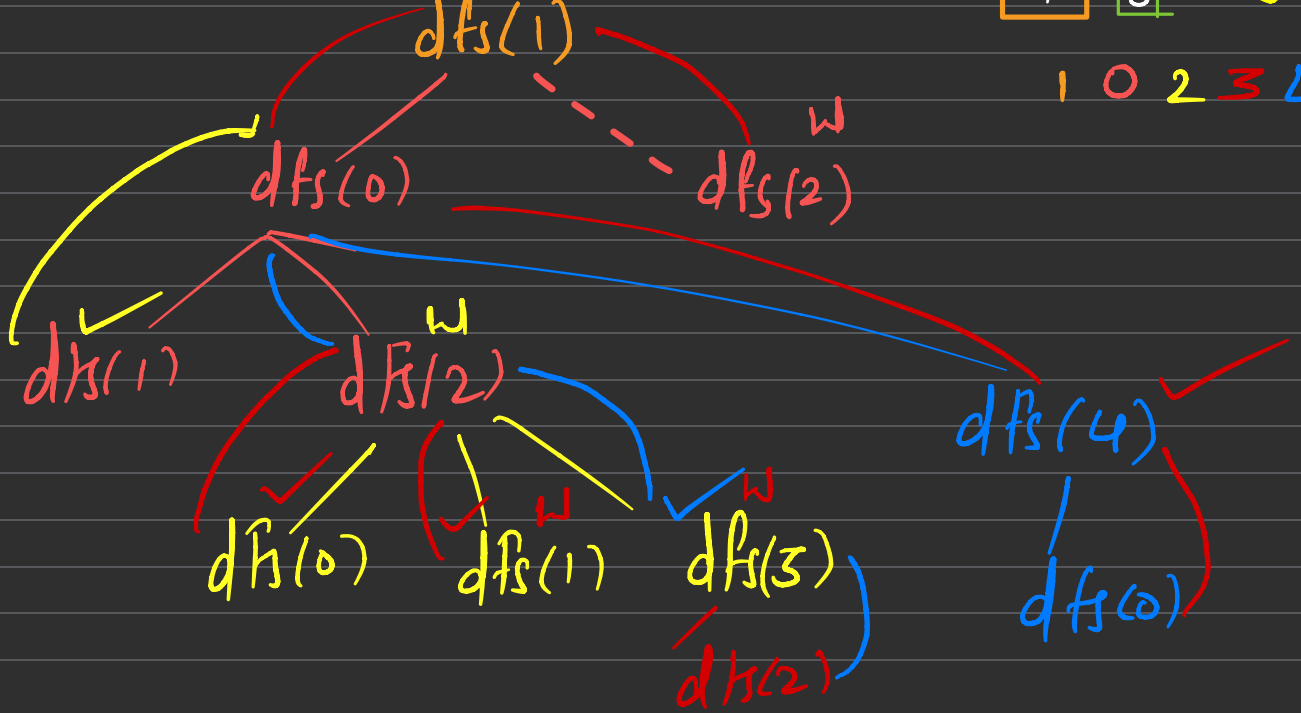
1 2 0



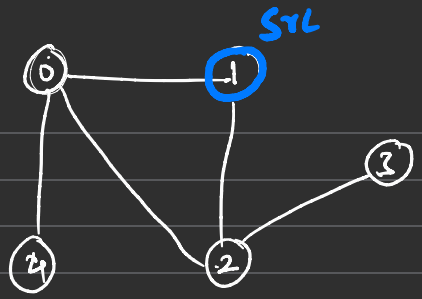




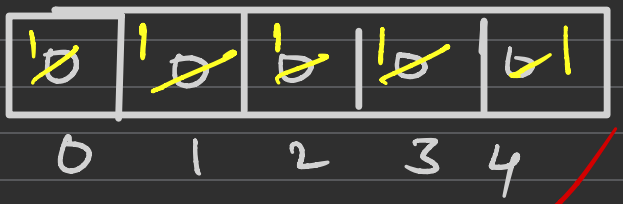
1 0 2 3 4

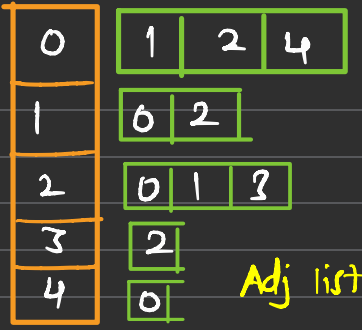


```
void DFS ( adjL , visit[] , ans[], src)
{
    ans.push(src)
    visit[src] = 1

    for (auto it : adjL[src])
    {
        if ( visit[it] == 0)
            DFS(adjL, visit, ans, it)
    }

}

DFS ( adj. visit, ans, 0)
```



1 0 2 3 4





dfs(1)

dfs(0)

dfs(2)

dfs(4)

dfs(3)

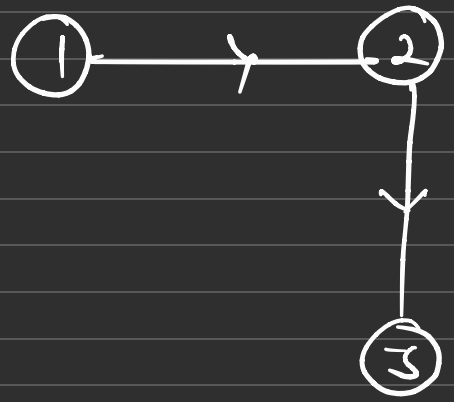
```
void DFS(vector<vector<int>>& adj, int visit[], vector<int> &ans, int src){

    visit[src] = 1;
    ans.push_back(src);

    for (auto adjNodes: adj[src]){

        if (visit[adjNodes] == 0){
            DFS(adj, visit, ans, adjNodes);
        }
    }
}
```

DFS(---,0)

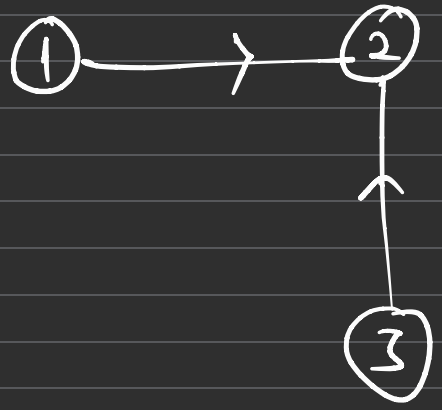
- Topological Sort

- linear ordering sorting technique which mean, if there is edge from $u \rightarrow v$, then in your sorting u should come before v

- Only applied on
  1. Directed graph
  2. Acyclic graph



1 2 3

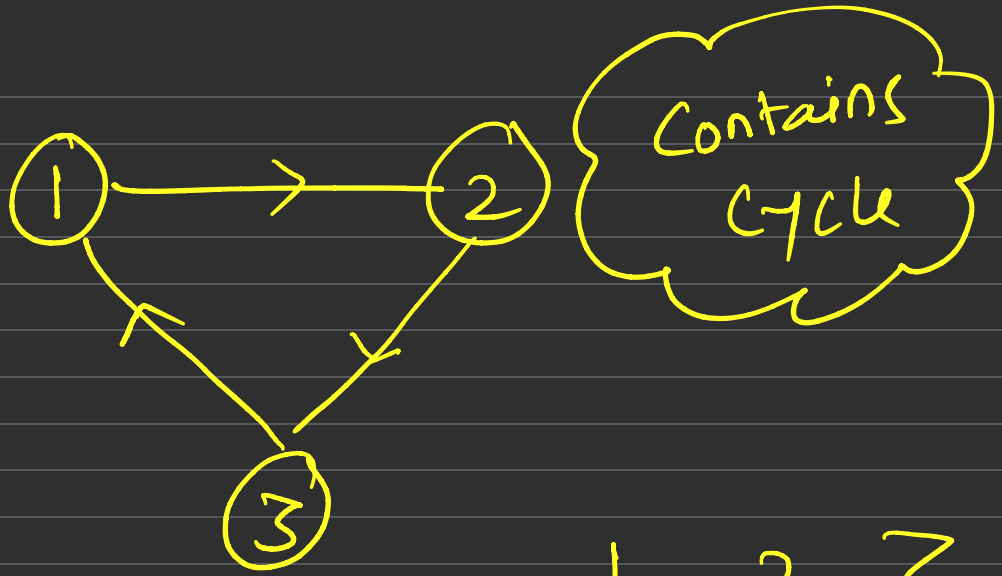


1 3 2

Why not on Undirected graph

1 2

2 1



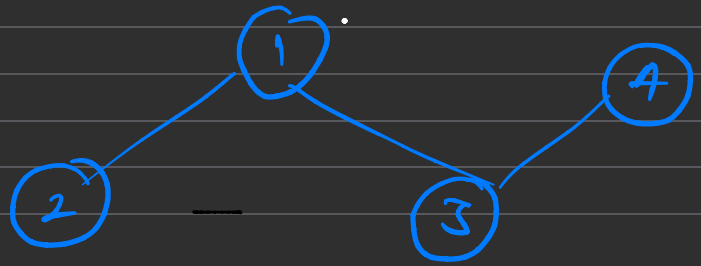
1 2 3

- Indegree (Directed)

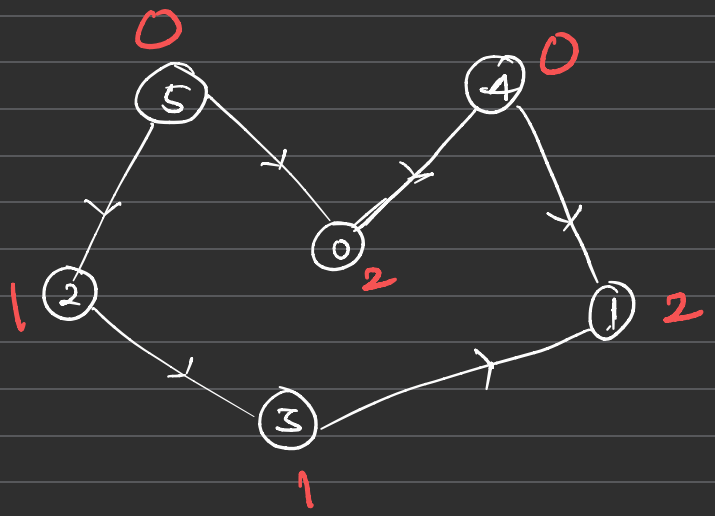
  No. of edges coming inside

- Outdegree (Directed)

- degree (Undirected)



- find topo. sort



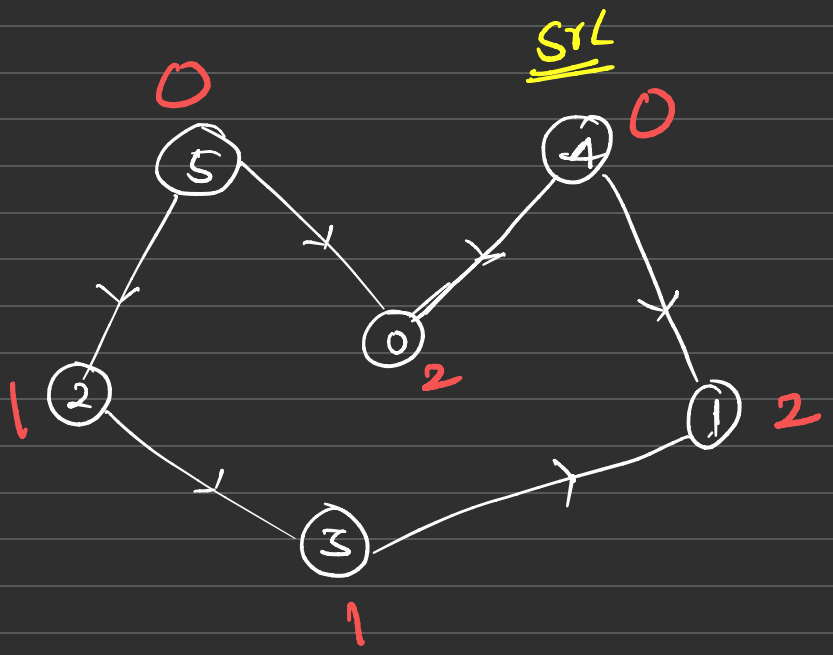
5 4 2 0 3 1

(1.) find indegree of all Node

(2.) Take a Node having mini. indegree, print it & Remove it's out going edges

(3.) Readjust indegree

(4) perform step (2)



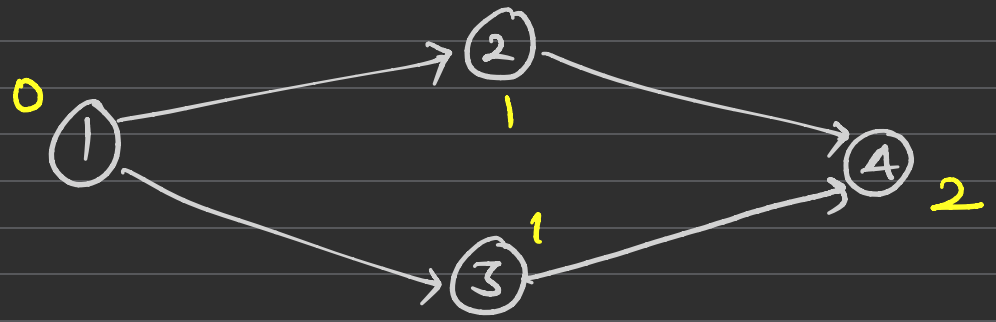
0 1 2 3

dfs(2)

dfs(3)

0	1	2	3
1 0	0 1	0 1	0 1

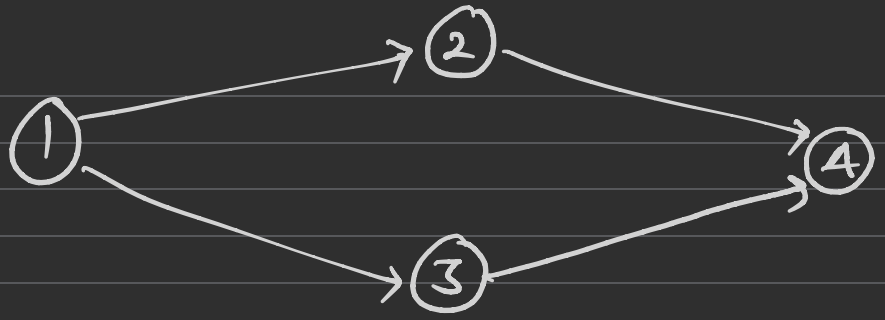
```
for (i=0 ; i<V ; i++)
  if (visited[i] == 0)
      DFS(i)
```



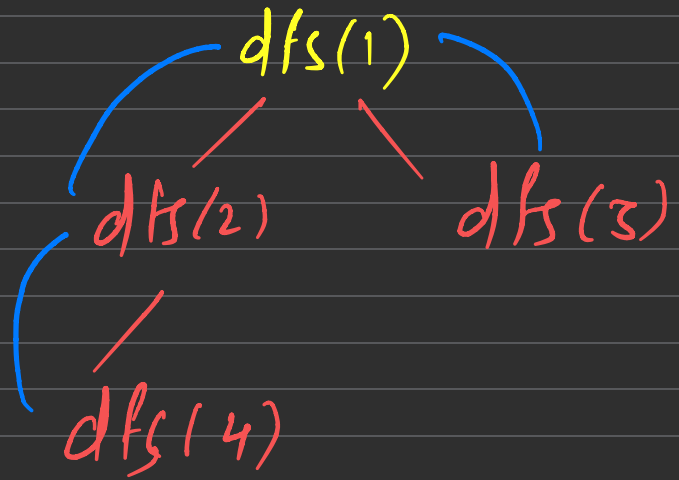
→ 1 2 3 4

→ 1 3 2 4

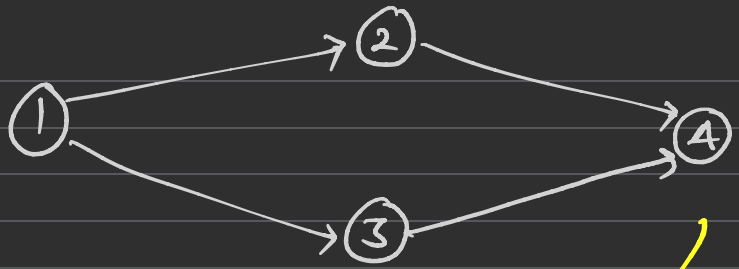
} Topo sort

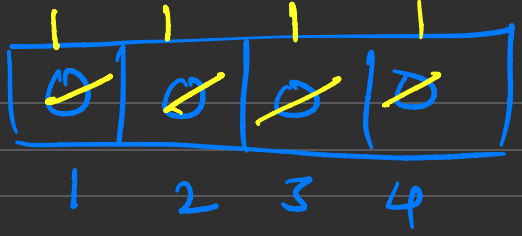


1	2	3	4
~~1~~ 0	~~0~~ 1	~~0~~ 1	~~0~~ 1



1 2 4 3





dfs(1)

dfs(2)

dfs(3)

dfs(4)

LIFO

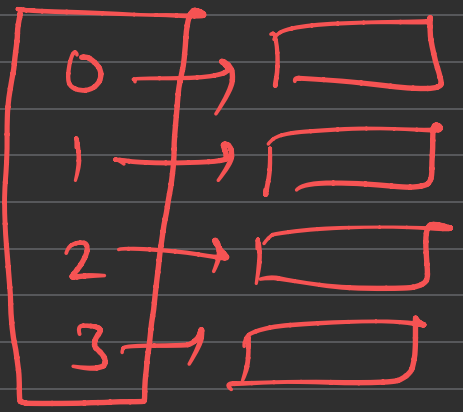
1
3
2
4

Stack

matrix [[3,0], [1,0], [2,0]] → [u, v]

u → v

```
for(i=0; i<n; i++)
{   u = edge(i)(0)
    v = edge(i)(1)
```

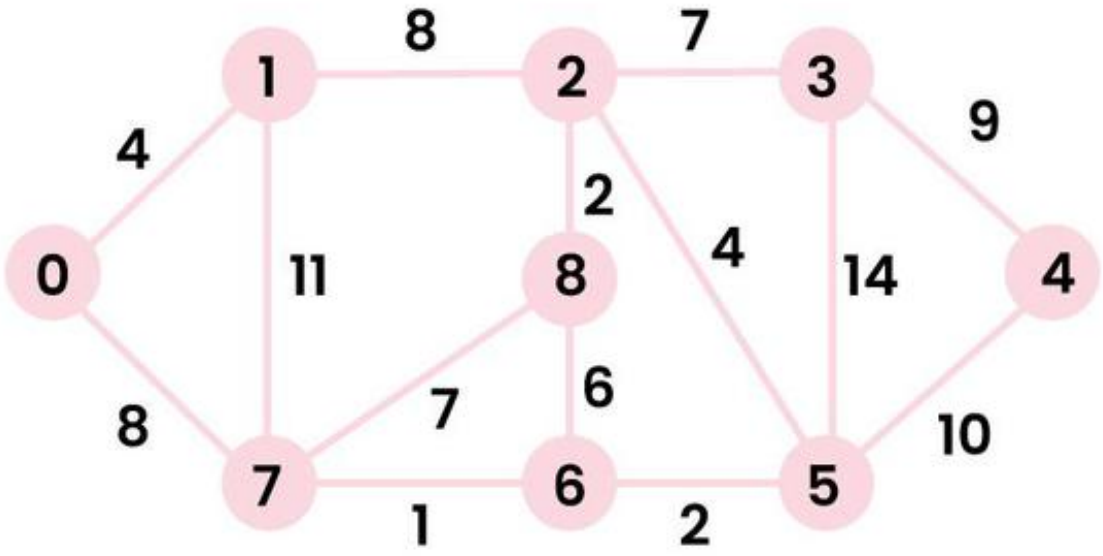


- Dijsktra's Algorithm:

  - Single Source Shortest path

— Negative edge weight → does not work

Dijsktra Algorithm



Working of Dijkstra's Algorithm